

## Лабораторна робота №2

### *Розділені різниці. Інтерполяційні многочлени Ньютона та факторіальних многочленів.*

**Мета:** вивчити методи обчислення розділених різниць довільного порядку і вивчити метод інтерполяції з використанням інтерполяційних многочленів Ньютона та факторіальних многочленів для довільної конфігурації вузлів.

#### Основні теоретичні відомості.

Введемо розділені різниці, які задовольняють наступним рекурентним співвідношенням. Розділена різниця нульового порядку співпадає зі значенням функції

$$f(x_i) = f(x_i).$$

Розділена різниця першого порядку

$$f(x_i, x_{i+1}) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}.$$

Розділена різниця другого порядку

$$f(x_i, x_{i+1}, x_{i+2}) = \frac{f(x_{i+1}, x_{i+2}) - f(x_i, x_{i+1})}{x_{i+2} - x_i}.$$

В загальному ми можемо записати рекурентну формулу для обчислення розділених різниць  $k$ -го порядку через розділені різниці  $(k-1)$  порядку:

$$f(x_i, \dots, x_{i+k}) = \frac{f(x_{i+1}, \dots, x_{i+k}) - f(x_i, \dots, x_{i+k-1})}{x_{i+k} - x_i}.$$

Можна показати, що

$$f(x_0, \dots, x_n) = \sum_{i=0}^n \frac{f(x_i)}{\prod_{\substack{j \neq i \\ j=0, n}} (x_i - x_j)}.$$

Доведення цього твердження зробимо методом математичної індукції. При  $n=0$  отримаємо очевидну тотожність  $f(x_0) = f(x_0)$ . Покажемо, що зі справедливості нашого твердження при  $n$  випливає його справедливості при  $n+1$ . Враховуючи властивості скінченних різниць, запишемо вираз

$$f(x_0, x_1, \dots, x_{n+1}) = \frac{f(x_1, \dots, x_{n+1}) - f(x_0, \dots, x_n)}{x_{n+1} - x_0}$$

Підставимо сюди вираз для розділеної різниці  $n$ -порядку:

$$f(x_0, x_1, \dots, x_{n+1}) = \frac{1}{x_{n+1} - x_0} \left( \sum_{i=1}^{n+1} \frac{f(x_i)}{\prod_{\substack{j \neq i \\ j=1, n+1}} (x_i - x_j)} - \sum_{i=0}^n \frac{f(x_i)}{\prod_{\substack{j \neq i \\ j=0, n}} (x_i - x_j)} \right)$$

Знайдемо значення коефіцієнта при  $f(x_i)$ . Окремо розглянемо випадки  $i = 1 \dots n$  та  $i = 0, i = n + 1$ .

У випадку  $i = 1 \dots n$  отримаємо

$$\begin{aligned} & \frac{1}{x_{n+1} - x_0} \left( \frac{1}{\prod_{\substack{j \neq i \\ j=1, n+1}} (x_i - x_j)} - \frac{1}{\prod_{\substack{j \neq i \\ j=0, n}} (x_i - x_j)} \right) = \\ & = \frac{1}{x_{n+1} - x_0} \left( \frac{(x_i - x_0) - (x_i - x_{n+1})}{\prod_{\substack{j \neq i \\ j=0, n+1}} (x_i - x_j)} \right) = \\ & = \frac{1}{\prod_{\substack{j \neq i \\ j=0, n+1}} (x_i - x_j)} \end{aligned}$$

Аналогічно для випадку  $i = 0$  отримаємо

$$\frac{1}{x_{n+1} - x_0} \left( - \frac{1}{\prod_{\substack{j \neq 0 \\ j=0, n}} (x_0 - x_j)} \right) = \frac{1}{\prod_{\substack{j \neq 0 \\ j=0, n+1}} (x_0 - x_j)}$$

Для випадку  $i = n + 1$  знаходимо, що

$$\frac{1}{x_{n+1} - x_0} \left( \frac{1}{\prod_{\substack{j \neq n+1 \\ j=1, n+1}} (x_{n+1} - x_j)} \right) = \frac{1}{\prod_{\substack{j \neq n+1 \\ j=0, n+1}} (x_{n+1} - x_j)}$$

Якщо підсумувати отримані результати, ми отримаємо, що розділена різниця  $(n + 1)$ -порядку визначається виразом

$$f(x_0, \dots, x_{n+1}) = \sum_{i=0}^{n+1} \frac{f(x_i)}{\prod_{\substack{j \neq i \\ j=0, n+1}} (x_i - x_j)}$$

який впливає з постульованого виразу для розділеної різниці  $n$ -порядку. Таким чином наше твердження доведено

Так як розділена різниця являє собою лінійну комбінацію значень функції у вузлах, тому значення розділеної різниці не змінюється при перестановці її аргументів.

Можна показати, що розділена різниця  $n$  – порядку по порядку величини визначається похідною від функції  $n$  – порядку (доведення цього твердження буде дане при розгляді чисельного диференціювання):

$$f(x_0, \dots, x_n) = \frac{f^{(n)}(\xi)}{n!}$$

де  $\xi \in [x_0, x_n]$

Розглянемо іншу форму запису інтерполяційного многочлена. Але спочатку виконаємо ряд проміжних перетворень. Знайдемо значення різниці

$$\begin{aligned} f(x) - L_n(x) &= f(x) - \sum_{i=0}^n f(x_i) \prod_{\substack{j \neq i \\ j=0, n}} \frac{x - x_j}{x_i - x_j} = \\ &= \prod_{i=0}^n (x - x_i) \left( \frac{f(x)}{\prod_{i=0}^n (x - x_i)} - \sum_{i=0}^n \frac{f(x_i)}{(x - x_i) \prod_{\substack{j \neq i \\ j=0, n}} (x_i - x_j)} \right) = \\ &= \prod_{i=0}^n (x - x_i) \left( \frac{f(x)}{\prod_{i=0}^n (x - x_i)} + \sum_{i=0}^n \frac{f(x_i)}{(x_i - x) \prod_{\substack{j \neq i \\ j=0, n}} (x_i - x_j)} \right) = \\ &= f(x, x_0, \dots, x_n) w_n(x) \end{aligned}$$

Обчислимо тепер різницю двох інтерполяційних многочленів Лагранжа  $L_n(x) - L_{n-1}(x)$ . Це є алгебраїчний многочлен степені  $n$ , який перетворюється в нуль у вузлах  $0, 1, \dots, n-1$ . Тому ми його можемо записати у вигляді

$$L_n - L_{n-1} = A_{n-1} w_{n-1}(x),$$

де  $A_{n-1}$  – деяка стала.

Якщо  $x = x_n$ , то значення інтерполяційного многочлена Лагранжа співпадає зі значенням функції в цьому вузлі  $L_n(x_n) = f(x_n)$ . Тому

$$f(x_n) - L_{n-1}(x_n) = A_{n-1} w_{n-1}(x_n)$$

З іншого боку, якщо ми використаємо результати попереднього випадку, то при  $x = x_n$  отримаємо

$$\begin{aligned} f(x_n) - L_{n-1}(x_n) &= f(x_n, x_0, x_1, \dots, x_{n-1}) w_{n-1}(x_n) = \\ &= f(x_0, x_1, \dots, x_{n-1}, x_n) w_{n-1}(x_n) \end{aligned}$$

Порівнявши ці два вирази, отримаємо значення константи  
 $A_{n-1} = f(x_0, x_1, \dots, x_n)$ . Таким чином

$$L_n(x) - L_{n-1}(x) = f(x_0, x_1, \dots, x_n) w_{n-1}(x).$$

Представимо інтерполяційний многочлен Лагранжа у вигляді тотожності:

$$L_n(x) = L_0(x) + (L_1(x) - L_0(x)) + (L_2(x) - L_1(x)) + \dots + (L_n(x) - L_{n-1}(x))$$

Врахуємо, що

$$L_0(x) = f(x_0)$$

$$L_1(x) - L_0(x) = f(x_0, x_1) w_0(x) = (x - x_0) f(x_0, x_1)$$

$$L_2(x) - L_1(x) = f(x_0, x_1, x_2) w_1(x) = (x - x_0)(x - x_1) f(x_0, x_1, x_2)$$

...

$$L_n(x) - L_{n-1}(x) = f(x_0, x_1, \dots, x_n) w_{n-1}(x) = (x - x_0)(x - x_1) \dots (x - x_{n-1}) f(x_0, x_1, \dots, x_n)$$

В результаті отримаємо представлення інтерполяційного многочлена Лагранжа через розділені різниці. Отриманий таким чином многочлен називається інтерполяційним многочленом Ньютона для інтерполяції вперед:

$$N_n(x) = f(x_0) + (x - x_0) f(x_0, x_1) + (x - x_0)(x - x_1) f(x_0, x_1, x_2) + \dots \\ \dots + (x - x_0)(x - x_1) \dots (x - x_{n-1}) f(x_0, x_1, \dots, x_n)$$

Аналогічно ми можемо отримати вираз для інтерполяційного многочлена Ньютона для інтерполяції назад

$$N_n(x) = f(x_n) + (x - x_n) f(x_n, x_{n-1}) + (x - x_n)(x - x_{n-1}) f(x_n, x_{n-1}, x_{n-2}) + \dots \\ \dots + (x - x_n)(x - x_{n-1}) \dots (x - x_1) f(x_n, x_{n-1}, \dots, x_0)$$

В процесі проміжних перетворень ми отримали вираз, який дає нам змогу оцінити точність інтерполяції

$$f(x) - L_n(x) = f(x, x_0, \dots, x_n) w_n(x)$$

Врахувавши, що для розділених різниць справедливо співвідношення, що

$$f(x_0, \dots, x_n) \approx \frac{f^{(n)}(\xi)}{(n+1)!}$$

де  $\xi \in [x_0, x_n]$ , отримаємо граничну оцінку величини похибки

$$R_n = w_n(x) \frac{M_{n+1}}{(n+1)!},$$

$$\text{де } M_{n+1} = \max_{x \in [x_0, x_n]} |f^{(n+1)}(x)|$$

Як ми бачимо інтерполяційний многочлен Ньютона має  $(n+1)$  порядок точності інтерполяції.

Поліном  $w_n(x)$  рівномірно обмежений по  $x$ . Тому справедлива оцінка:

$$|w_n(x)| < (nh)^{n+1}$$

Це нам дає змогу для отримання заданої точності інтерполяції при фіксованому числі вузлів визначити крок сітки з умови

$$(nh)^{n+1} \leq \frac{\varepsilon(n+1)!}{M_{n+1}}$$

Використовувати формулу Стірлінга для оцінки значення  $n!$

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n,$$

ми можемо оцінити число вузлів при відомому кроці сітки необхідне для отримання заданої точності

$$\sqrt{\frac{2}{\pi n}} \cdot M_{n+1} \cdot \left(\frac{h}{2}\right)^{n+1} < \varepsilon$$

При інтерполяції на рівномірній сітці потрібно вибирати з таблиці вузли так, щоб задане значення  $x$  потрапляло у центр конфігурації розміщення цих вузлів.

Якщо ми хочемо інтерполювати функцію  $f(x) = \sin x$ , то для досягнення точності  $\varepsilon = 5 \cdot 10^{-7}$  у випадку  $n=1$  крок має задовольнити умові  $h \leq 0.002$ . У випадку  $n=2$  для досягнення цієї ж точності, отримаємо оцінку величини кроку  $h \leq 0.02$

## Факторіальні многочлени

У випадку рівновіддалених вузлів для задачі інтерполяції зручно використовувати скінченні різниці. Нехай ми маємо сукупність

рівновіддалених вузлів  $\{x_i, f_i\}$ ,  $i = 0 \dots n$ . Виконаємо заміну змінних  $t = \frac{(x - x_0)}{h}$ .

Отримаємо, що в нових координатах координати вузлів сітки  $t = 0, 1, \dots, n$ . Крок сітки рівний одиниці.

### Різницевий оператор.

Основним оператором, в обчисленні скінченних різниць, є різницевий оператор, який визначається рівністю:

$$\Delta f(t) = f(t+1) - f(t)$$

Розглянемо його основні властивості:

Різницевий оператор є лінійним, тобто

$$\Delta(\alpha f(t) + \beta g(t)) = \alpha \Delta f(t) + \beta \Delta g(t)$$

Дія різницевого оператора на добуток функцій дає

$$\Delta(f(t)g(t)) = f(t+1)g(t+1) - f(t)g(t) =$$

$$f(t+1)g(t+1) - f(t+1)g(t) + f(t+1)g(t) - f(t)g(t) =$$

$$f(t+1)(g(t+1) - g(t)) + g(t)(f(t+1) - f(t)) =$$

$$f(t+1)\Delta g(t) + g(t)\Delta f(t)$$

Дія різницевого оператора на частку двох функцій дає:

$$\Delta \left( \frac{f(t)}{g(t)} \right) = \frac{f(t+1)}{g(t+1)} - \frac{f(t)}{g(t)} = \frac{g(t)\Delta f(t) - f(t)\Delta g(t)}{g(t+1)g(t)}$$

Дія різницевого оператора на деякі стандартні функції визначається як

$$\Delta a^t = a^t (a - 1)$$

$$\Delta 2^t = 2^t (2 - 1)$$

$$\Delta \ln(t) = \ln \left( 1 + \frac{1}{t} \right)$$

$$\Delta \sin(at + b) = 2 \sin \left( \frac{a}{2} \right) \cos \left( a \left( t + \frac{1}{2} \right) + b \right)$$

$$\Delta \cos(at + b) = -2 \sin \left( \frac{a}{2} \right) \sin \left( a \left( t + \frac{1}{2} \right) + b \right)$$

Різницевий оператор можна підносити до степені  $\Delta^2 f(t) = \Delta(\Delta f(t))$ .

В загальному випадку можна записати:  $\Delta^n f(t) = \Delta(\Delta^{n-1} f(t))$

Розглянемо основну теорему в обчисленні різниць:

Для алгебраїчного многочлена  $f(t) = a_0 + a_1 t + \dots + a_n t^n$  різниця  $n$ -го порядку рівна  $a_n n!$ , а різниця  $(n+1)$  порядку – рівна нулю.

Якщо  $f(t)$  - алгебраїчний многочлен степені  $n$ , то  $\Delta f(t)$  є алгебраїчний многочлен степені  $n-1$ . Щоб довести це твердження, розглянемо випадок  $f(t) = t^n$  :

$$\begin{aligned} \Delta(t^n) &= (t+1)^n - t^n = \sum_{k=0}^n C_n^k t^k - t^n = \\ &= t^n + n t^{n-1} + \dots + 1 - t^n = n t^{n-1} + \dots \end{aligned}$$

Подіавши різницевим оператором на алгебраїчний многочлен  $f(t) = a_0 + a_1 t + \dots + a_n t^n$   $n$  раз, ми отримаємо необхідний результат  $a_n n!$ .

### Оператор зсуву

Введемо оператор зсуву за допомогою співвідношення

$$E(f(t)) = f(t+1)$$

Розглянемо тепер його основні властивості:

Оператор зсуву є лінійним

$$E(\alpha f(t) + \beta g(t)) = \alpha E(f(t)) + \beta E(g(t))$$

Оператор зсуву можна підносити до степені  $E^n(f(t)) = f(t+n)$

$$E^0(f(t)) = f(t)$$

$$E^{-n}(f(t)) = f(t-n)$$

Між різницевим оператором і оператором зсуву є співвідношення  $\Delta = E - 1$ . Його правильність впливає з наступного:

$$\Delta f(t) = f(t+1) - f(t) = E(f(t)) - f(t) = (E - 1)(f(t))$$

Використовуючи оператор зсуву, доведемо правильність формули для обчислення скінченних різниць:

$$\Delta^n = (E - 1)^n = \sum_{k=0}^n (-1)^{n-k} C_n^k E^k$$

Отримаємо:

$$\Delta f(t) = \sum_{k=0}^n (-1)^{n-k} C_n^k E^k (f(t)) = \sum_{k=0}^n (-1)^{n-k} C_n^k f(t+k)$$

### Факторіальні многочлени

В математичному аналізі важливу роль відіграє функція  $x^n$ . Це зумовлено тим, що довільна функція може бути розкладена в ряд по степенях  $x^n$ .

$$f(x) = \sum_{n=0}^{\infty} a_n x^n$$

Можна показати, що ця властивість функцій  $x^n$  в першу чергу визначається тим, що перша похідна задовольняє співвідношення:

$$(x^n)' = nx^{n-1}$$

В чисельних методах важливу роль відіграють функції – факторіальні многочлени. Факторіальні многочлени вводять виходячи з умови

$$\Delta t^{(n)} = nt^{(n-1)}$$

Цій умові задовольняють наступні многочлени  $t^{(n)} = t(t-1)\dots(t-n+1)$ .

Факторіальні многочлени задовольняють наступним властивостям:

$t^{(0)} = 1$ . Звідси випливає, що  $0^{(0)} = 1$ . Це відповідає такому формальному співвідношенню  $0! = 1$ .

Факторіальний многочлен  $t^{(n)}$  має  $n$  множників.

Доведемо основну властивість факторіальних многочленів.

$$\begin{aligned} \Delta t^{(n)} &= (t+1)^{(n)} - t^{(n)} = \\ &= (t+1)t(t-1)\dots(t-(n-2)) - t(t-1)\dots(t-(n-1)) = \\ &= t(t-1)\dots(t-(n-2))(t+1-t+n-1) = \\ &= nt(t-1)\dots(t-(n-2)) = nt^{(n-1)} \end{aligned}$$

Доведемо, що довільну функцію можна розкласти в ряд по факторіальних многочленах.

$$f(t) = \sum_{k=0}^{\infty} b_k t^{(k)}$$

При  $t=0$  знаходимо, що  $f(0) = b_0$ . Подіємо різницеvim оператором на обидві сторони рівняння  $\Delta f(t) = \sum_{k=1}^{\infty} b_k k t^{(k-1)}$ . Аналогічно знаходимо, що при  $t=0$   $\Delta f(0) = b_1$ .

Подіавши різницеvim оператором  $n$  раз отримаємо:

$$\Delta^n f(t) = \sum_{k=n}^{\infty} b_k k(k-1)\dots(k-(n-1)) t^{(k-n)}$$

При  $t=0$   $\Delta^n f(0) = n! b_n$ . Отже, ми отримали ряд по факторіальних многочленах:

$$f(t) = \sum_{k=0}^{\infty} \frac{\Delta^k f(0)}{k!} t^{(k)}$$

Очевидно, що у випадку скінченної кількості вузлів ми повинні обривати наш факторіальний ряд в залежності від числа вузлів.

### Загальний хід роботи

1. Скласти програму для табуляції заданої даних (*варіанти нижче відповідно по списку в групі, б студент по порядковому номері списку підгрупи бере 1 варіант і т.д.*) на відрізку  $[x_0, x_n]$  з кроком  $h = \frac{x_n - x_0}{n}$ , де  $n$  - задане число вузлів. Результати табуляції записати в текстовий файл. Отриману сукупність вузлів  $\{x_i, y_i\}$  використати в подальшому як набір вхідних даних.
2. Скласти програму знаходження довільного значення функції  $f(x)$  за допомогою інтерполяційного многочлена Ньютона. В програмі записати наступні функції: зчитування вхідних даних з текстового файлу – масивів  $x_i, f_i, i = 0 \dots n$ , знаходження значення функції  $\omega_k(x) = \prod_{i=0}^k (x - x_i)$ , знаходження значення розділеної різниці довільного порядку  $k$   $f(x_0, \dots, x_k) = \sum_{i=0}^k \frac{f(x_i)}{\prod_{\substack{j \neq i \\ j=0, k}} (x_i - x_j)}$ , знаходження значення інтерполяційного многочлена Ньютона  $N_n(x) = f_0 + \sum_{k=1}^n \omega_{k-1} f(x_0, \dots, x_k)$ , обчислення похибки інтерполяції многочленами Ньютона  $\varepsilon(x) = |f(x) - N_n(x)|$ .
3. Протабулювати на відрізку  $[a, b]$  з кроком  $h = \frac{b-a}{20n}$  задану функцію  $f(x)$ , інтерполяційний многочлен Ньютона  $N_n(x)$ , похибку  $\varepsilon(x)$  і функцію  $w_n(x)$ . Побудувати графіки цих функцій.
4. Дослідити точність інтерполяції в залежності від кроку  $h$  (різне число вузлів  $n = 5, 10, 20$  при заданому інтервалі  $[a, b]$ ) та від числа вузлів (постійний крок  $h$ , різний інтервал  $[a, b]$ , де  $b = a + hn$ ,  $n = 5, 10, 20$ ).
5. Результати виконання лабораторної роботи оформити у вигляді звіту.



### Приклад CSV-файлу *data.csv*

```
n,t
1000,3
2000,5
4000,11
8000,28
16000,85
```

### Код для зчитування даних з CSV файлу:

```
import csv

def read_data(filename):
    x = []
    y = []

    with open(filename, 'r', newline='') as file:
        reader = csv.DictReader(file)
        for row in reader:
            x.append(float(row['n']))
            y.append(float(row['t']))

    return x, y

# Використання
x, y = read_data("data.csv")

print("x:", x)
print("y:", y)
```

## ВАРІАНТ 1. Інженерія продуктивності. Прогнозування часу виконання алгоритму

### *Постановка задачі*

У програмному забезпеченні важливим є дослідження залежності часу виконання алгоритму від розміру вхідних даних. Під час розробки програмного забезпечення необхідно оцінити, як змінюється продуктивність системи при зростанні навантаження, щоб забезпечити її масштабованість та ефективність. У процесі профілювання програми проводяться експериментальні вимірювання часу виконання алгоритму для різних значень розміру вхідних даних  $n$ .

*За результатами профілювання алгоритму отримано експериментальні дані:*

| <b>n (розмір вхідних даних)</b> | <b>t (мс)</b> |
|---------------------------------|---------------|
| 1000                            | 3             |
| 2000                            | 5             |
| 4000                            | 11            |
| 8000                            | 28            |

Необхідно:

1. Побудувати інтерполяційний многочлен Ньютона
2. Спрогнозувати час виконання при  $n = 6000$
3. Дослідити точність прогнозу при зміні кількості вузлів

Хід роботи

1. Зчитати дані з CSV-файлу (приклад зчитування вище)
2. Побудувати таблицю розділених різниць
3. Обчислити  $P_n(6000)$  (методами Ньютона і факторіальними многочленами)
4. Побудувати графік:
  - експериментальні точки
  - інтерполяційна крива
5. Повторити обчислення:
  - для 5 вузлів
  - для 10 вузлів
  - для 20 вузлів
6. Проаналізувати зміну похибки

## ВАРІАНТ 2. Планування обчислювальних ресурсів (DevOps). Прогнозування навантаження на CPU (графічний процесор)

Постановка задачі

У сучасних хмарних інфраструктурах, зокрема на платформах типу Amazon Web Services, Microsoft Azure або Google Cloud, важливим завданням є планування обчислювальних ресурсів (Capacity Planning). Одним із ключових показників є завантаження процесора (CPU), яке безпосередньо залежить від рівня навантаження на систему наприклад, кількості запитів за секунду (RPS, requests per second).

У процесі експлуатації сервісу збираються історичні дані про залежність між RPS і відсотком використання CPU.

На основі історичних даних навантаження:

| RPS | CPU (%) |
|-----|---------|
| 50  | 20      |
| 100 | 35      |
| 200 | 60      |
| 400 | 110     |
| 800 | 210     |

Потрібно:

1. Побудувати модель  $CPU = f(RPS)$
2. Оцінити CPU при 600 RPS
3. Дослідити вплив кількості вузлів на прогноз

*Хід роботи*

1. Зчитати дані з CSV-файлу (*приклад зчитування вище*)
2. Побудувати таблицю розділених різниць
3. Обчислити прогноз для 600 RPS (методами Ньютона і факторіальними многочленами)
4. Побудувати графік CPU(RPS)
5. Дослідити:
  - інтерполяцію при 5, 10 та 20 вузлах
  - похибку при цій кількості вузлів
6. Зробити висновок щодо стабільності моделі

### **ВАРІАНТ 3. Прогноз часу тренування моделі машинного навчання**

*Постановка задачі*

У задачах машинного навчання час тренування моделі залежить від низки факторів, зокрема від розміру навчального набору даних, складності самої моделі, доступних обчислювальних ресурсів (CPU або GPU), а також обраних параметрів навчання. У практиці MLOps, під час роботи з такими фреймворками, як TensorFlow або PyTorch, часто виникає потреба заздалегідь оцінити тривалість навчання моделі. Це необхідно для того, щоб визначити, чи встигне модель натренуватися до встановленого дедлайну, скільки графічних процесорів потрібно зарезервувати, а також чи є доцільним збільшення обсягу навчальних даних.

Повторне повне тренування моделі для кожного нового розміру датасету є недоцільним, оскільки воно потребує значних обчислювальних ресурсів, займає тривалий час і призводить до додаткових фінансових витрат. Саме тому на основі вже отриманих експериментальних результатів доцільно побудувати інтерполяційну модель, яка дозволяє оцінювати час тренування для проміжних значень розміру вибірки без повторного запуску процесу навчання. Такий підхід забезпечує швидке прогнозування та ефективне планування ресурсів у реальних ML-проектах.

*Експериментальні дані:*

| Dataset size | Train time (sec) |
|--------------|------------------|
| 10000        | 8                |
| 20000        | 20               |
| 40000        | 55               |
| 80000        | 150              |

|        |     |
|--------|-----|
| 160000 | 420 |
|--------|-----|

Необхідно:

1. Побудувати інтерполяційну модель
2. Оцінити час для 120000 (методами Ньютона і факторіальними многочленами)
3. Дослідити ефект Рунге при збільшенні вузлів

*Хід роботи*

1. Зчитати дані з CSV-файлу (приклад зчитування вище)
2. Обчислити прогноз (методами Ньютона і факторіальними многочленами)
3. Побудувати графік
4. Збільшити кількість вузлів ( $n=5, 10, 20$ ) та побудувати графік похибок при зазначених кількостях вузлів.
5. Проаналізувати коливання полінома

#### **ВАРІАНТ 4. Прогноз вартості обчислень та ресурсів у хмарній інфраструктурі**

*Постановка задачі*

У хмарних середовищах механізм автоматичного масштабування (autoscaler) повинен у реальному часі визначати: скільки контейнерів або віртуальних машин необхідно запустити; яке очікуване навантаження на CPU; якою буде приблизна вартість обчислень. Приклад практичної ситуації: система отримує 600 RPS (requests per second/запитів в секунду) і необхідно оцінити: прогнозоване використання CPU  $\rightarrow$  CPU(600) відповідну кількість контейнерів приблизну вартість обробки

| Tasks | Cost (\$) |
|-------|-----------|
| 1000  | 0.2       |
| 2000  | 0.45      |
| 5000  | 1.5       |
| 10000 | 3.2       |
| 20000 | 7.5       |

Потрібно:

1. Побудувати модель  $Cost = f(tasks)$
2. Обчислити прогноз для 15000
3. Оцінити похибку

*Хід роботи*

1. Зчитати дані з CSV-файлу (приклад зчитування вище)

2. Побудувати таблицю розділених різниць
3. Реалізувати інтерполяцію (методами Ньютона і факторіальними многочленами)
4. Побудувати графік
5. Порівняти результати та похибки для різної кількості вузлів ( $n=5, 10, 20$ )
6. Проаналізувати стабільність моделі

## **ВАРІАНТ 5. Оптимізація ігрового рушія. Прогнозування FPS**

(на прикладі сучасних рушіїв, зокрема Unity Technologies та Epic Games, що розробляє Unreal Engine)

### *Постановка задачі*

У сучасних ігрових рушіях система LOD (Level of Detail) відповідає за автоматичне регулювання рівня деталізації об'єктів сцени з метою забезпечення стабільної продуктивності.

Одним із ключових показників продуктивності є FPS (Frames Per Second) кількість кадрів за секунду. Для комфортної гри FPS не повинен бути меншим за 60 кадрів/с.

Оскільки один кадр повинен обчислюватися приблизно за *16 мс*, повний перерахунок сцени при кожній зміні кількості об'єктів є обчислювально дорогим і неприйнятним у режимі реального часу.

| <b>Objects</b> | <b>FPS</b> |
|----------------|------------|
| 100            | 120        |
| 200            | 110        |
| 400            | 90         |
| 800            | 65         |
| 1600           | 40         |

### *Необхідно:*

1. Побудувати модель  $FPS = f(n)$
2. Оцінити FPS для 1000 об'єктів
3. Визначити мінімальну кількість об'єктів для  $FPS \geq 60$

### *Хід роботи*

1. Зчитати дані з CSV-файлу (*приклад зчитування вище*)
2. Побудувати таблицю розділених різниць
3. Обчислити прогноз (методами Ньютона і факторіальними многочленами)
4. Побудувати графік  $FPS(n)$
5. Дослідити вплив кількості вузлів ( $n=5, 10, 20$ ) та побудувати графік похибок для зазначених кількості вузлів.
6. Зробити висновки щодо оптимізації

## Дослідницька частина (для всіх варіантів)

Обов'язково виконати:

1. Дослідження впливу кроку  
Фіксований інтервал, різна кількість вузлів.
2. Дослідження впливу кількості вузлів, похибок.  
Фіксований крок, змінний інтервал.
3. Аналіз ефекту Рунге
4. Порівняння з методом Лагранжа (за бажанням для додаткових балів)

## Контрольні питання:

1. Що називається інтерполяцією? У чому її відмінність від екстраполяції?
2. У яких випадках застосовується інтерполяційний многочлен?
3. Що таке вузли інтерполяції?
4. Яка максимальна степінь інтерполяційного многочлена при  $n + 1$  вузлах?
5. У чому полягає метод Ньютона?
6. Що таке розділена різниця нульового, першого та вищих порядків?
7. Як формується таблиця розділених різниць?
8. Запишіть загальний вигляд інтерполяційного многочлена Ньютона.
9. Які переваги має форма Ньютона порівняно з формою Лагранжа?
10. Чому форма Ньютона зручна при додаванні нових вузлів?
11. Які основні етапи обчислення факторіальних многочленів?
12. У чому різниця між факторіальними многочленами та многочленами Ньютона?